
Q3.1 — "Your mentor asked for 'if the image is taken it should tell everything about that image.' Slide 2 implies you do it. Do you?"

Think. No. And the reason is specific and checkable, so give it precisely rather than calling it future work.

Answer. No. `POST /catalog/prefill` is the one route in `ai/service.py` still raising `NotImplementedError` (line 290). It is the only one left.

The reason is not that we ran out of time. `deepseek/deepseek-v4-flash` — the model behind the voice interpretation — **is a text model with no vision capability**. Handing it an image would at best be ignored and at worst be billed. So today **no image is sent to any model, anywhere in the system** (`docs/app/AI-Data-Flow.md`, "Never sent").

What actually happens on the voice screens, and this is a UI decision people mistake for a data flow: the photograph is on screen next to each question **purely so the human has something to look at while describing it**. The model receives the question id and the transcript. Never the image. There is a note in `CatalogVoice.jsx` saying exactly this, because it is the kind of thing someone assumes wrongly six months later.

What is already built for it. The seam exists — `ai/contracts.md` specifies `POST /catalog/prefill` as image in, the same listing fields out, **all nullable**, so the artisan *corrects* by voice rather than describing from scratch. Only the implementation is missing.

Four steps to turn it on, and step 3 is the one that matters:

1. Choose a model with vision — a **deliberate second model id**, not a swap of the constant in `interpret.py`, because the text interpreter should stay cheap and fast.
2. Implement `catalog_prefill` in `ai/service.py`.
3. **Extend the allowlist deliberately.** `build_payload()` permits four fields and an image is not one of them. **That refusal is the safety property.** Give vision its own payload builder with its own allowlist rather than loosening the text one.
4. Update `AI-Data-Flow.md` — an image is a new *kind* of data and that table is meant to be complete.

The blocker that is not engineering, and we would rather raise it than be asked: a photograph of a product taken inside someone's home **contains their home** — faces, children, the inside of a house. **The artisan consented to a marketplace listing. That is not consent to a foreign inference provider.** EXIF stripping (already unconditional, `app/src/api/upload.ts`, re-encoded through a canvas) protects against home GPS reaching a public listing; it decides nothing about whether the pixels may go to a third party. The `/consent` notice would have to say so, and provider retention has to be answered **before** shipping, not after.

Breaks when. Until it ships, the artisan describes from scratch instead of correcting — six voice questions instead of one confirmation. That is the friction the mentor asked us to remove and it is still there.

Q3.2 — "How much of your app depends on Bhashini, and have you actually got access?"

Think. Do not claim a government integration we have not confirmed. The strong move is to show the fallback ladder, because that is what makes the dependency survivable.

Today. Bhashini is the intended ASR/translation/TTS layer and `docs/decisions.md #5` is open: "ULCA portal self-serve keys, free prototyping tier — confirm commercial terms." `docs/Master-Technical-Reference.md §6.6` carries an explicit **[warning] UNVERIFIED** on access terms, rate limits and use-case eligibility, and `research/asr-bhashini/` is an empty directory with a `.gitkeep`. **We have not confirmed it. Saying otherwise would be the easiest thing in this deck to check.**

Why we want it anyway: government-aligned stack, better Indian dialect coverage, dramatically lower cost. On a MoSJE problem statement, using MeitY's own language mission is the correct architecture *and* the correct politics.

What makes the dependency survivable — the degradation ladder, which is built:

Layer	What runs	State
1	<code>stripCarrier()</code> + <code>matchCraft()</code> on-device, three languages, offline, free	Built. Runs <i>first</i> , answer used immediately when confident
2	<code>POST /catalog/interpret</code> → LLM, one turn in, one JSON out	Built
3	<code>compose_fallback</code> — builds the listing from the artisan's own raw answers, marks confidence: 0	Built
4	Visual icon grid — no language at all	Built

OPENROUTER_API_KEY unset is a supported state, not a failure. `/catalog/interpret` answers 503, the app falls back to its local tables, then to the grid, and onboarding completes either way. "Nothing in the app is gated on a model being reachable" — because our users are offline often, and a feature that only works with a working network is a feature that does not work.

TTS is deliberately not a live dependency. Web Speech API for Indic TTS is on the **rejected** list in `CLAUDE.md` — voice availability across Android handsets is unreliable. **Pre-generated audio instead.** That converts speech output from a per-request runtime dependency into a build-time asset: fixed cost, no rate limit, no latency, works offline.

At scale. Two ASR providers behind one interface, chosen per language, with measured WER per language and per dialect deciding the routing. Cost consequences are in the costing document — Bhashini free vs a commercial API is roughly the difference between ₹0 and ₹2.20 per listing.

Breaks when. If Bhashini's free tier does not cover production volume and commercial terms are unfavourable, ASR becomes our largest per-listing variable cost. That is a budget risk, not an architecture risk — the interface does not change.

Q3.3 — "The PS says descriptions in English AND Hindi. What if the artisan speaks Sambalpuri, not standard Odia?"

Think. Two separate problems get conflated here — dialect at input, and language pair at output. Separate them.

Output side — settled and unconditional. Both, always (docs/Master-Technical-Reference.md §6.1).

POST /catalog writes the listing once and ai/catalog/seo.py — **pure and deterministic** — cuts it to each channel's real character limits. Deterministic matters: the same listing renders the same way for GeM and for Amazon every time, so a rejected listing is reproducible instead of a coin flip.

Input side — the honest one. Our language set today is hi / or / en (ai/interpret.py; anything else becomes hi). Which languages ship at launch is docs/Master-Technical-Reference.md §17 open decision #8, still open.

Dialect is where ASR is weakest and where a general commercial model is weakest of all. A Sambalpuri speaker is not an edge case in this PS — Sambalpuri weaving is one of the flagship crafts.

Three things that reduce the damage, and all three are built:

1. **The local tier runs before the model** and handles ordinary phrasings by table, not by understanding. Dialect variation in "*mera naam ... hai*" is small; the table covers it.
2. **The artisan confirms every interpretation aloud.** "*Sambalpuri saree lag rahi hai, cotton ki. Sahi hai?*" → yes/no. **That confirmation is the real backstop and it must never be optimised away.**
3. **Free-text answers survive verbatim.** compose_fallback uses the artisan's own words when the model is unreachable or unsure. A mistranscribed dialect word does not become a wrong category — it becomes an unstructured phrase the human then corrects.

At scale. Per-dialect WER measurement, with a field recording set, before adding a language to the launch list.

Adding a language is not a code change — it is an ASR route, a translation pair, and a pack of pre-generated TTS strings.

Breaks when. A craft term with no standard spelling. *Bandha*, *ikat*, *bomkai* transcribe several ways and the category mapping downstream is doing fuzzy matching on the result. This is exactly where a **craft-term lexicon** would earn its keep, and we have not built one.

Q3.4 — "SEO-friendly, you said. How do you know your descriptions are SEO-friendly?"

Think. "SEO-friendly" is the softest phrase on the slide. Convert it into something measurable or concede it.

Answer — what is real. ai/catalog/seo.py is **pure and deterministic** and enforces each channel's actual constraints: title length, description length, attribute presence, keyword placement within the limits the channel will

accept. That is not SEO; **that is schema compliance, and it is the part that decides whether a listing is accepted at all.**

What is not real: we have no ranking measurement. We do not A/B test, we have no impression or click data, and nobody has verified that our descriptions rank better than an artisan's own. On a marketplace we do not control, we could not measure it if we tried.

The one place the claim is genuinely testable is our own marketplace, which is **SSR (Next.js) specifically so the catalog pages are indexable** (`docs/decisions.md`; `/p/:slug` is named as the SEO surface). There we can measure indexation and organic arrival, and that is the only honest SEO evidence we will ever have.

The stronger claim to make instead, because it is measurable and it is the thing that actually fails: **rejection-driven design**. `docs/Master-Technical-Reference.md` §5.3 lists the marketplace rejection triggers and each pipeline stage is built backwards from one of them — background not pure white, product under 85% of frame, under 1000×1000 (below which **zoom is disabled, which directly hurts conversion**), text or watermark on the main image, shadows, product shown in packaging, over 10 MB.

■ *"Put this mapping in the pitch — each rejection reason → the pipeline stage that prevents it."*

Breaks when. A judge asks for evidence a description ranks. We have none, and we should say "we can measure it on our own surface and nowhere else" rather than assert.

Q3.5 — "Your voice endpoint calls a paid model. What stops one artisan, or one bug, from spending your entire budget?"

Think. This is a cost-DoS question dressed as an engineering question. We have a real gap here and it is written in our own docs.

Today — the gap, stated plainly. `docs/app/Future-Implementations.md` §5: **POST /catalog/interpret is not rate-limited.** It is authenticated, so it is not an open relay, but *"an artisan id is currently used only for identification. It spends money per call."*

What limits the damage today, and it is more than nothing:

- **The local tier usually wins.** `stripCarrier()` and `matchCraft()` answer the ordinary case on-device, for free. **The common case sends nothing at all.**
- **Bounded call sites.** The model is reached from exactly three places, each with a ceiling: `onboard.name` once per artisan (and only if the local strip fails), `onboard.craft` only via "something else", and `catalog.q_*` at most five per product.
- **KNOWN_QUESTIONS refuses anything else with a 422** — deliberately, so that adding a question is a decision rather than a drift.
- **temperature: 0, no tools, no URLs, no ids.** The model returns a string.

At scale — and this is standard, not clever: per-artisan and per-tenant token buckets in Redis, a hard daily spend

ceiling per artisan, a global circuit breaker on spend velocity, and alerting on cost per listing rather than on request count. The generative image path already has the pattern we should copy: **opt-in, hard-capped, 2 free per artisan or unlocked after first sale** — *"otherwise one enthusiastic user burns the budget."*

Breaks when. A retry loop on a flaky network. Today nothing distinguishes fifty genuine questions from one screen retrying fifty times. That is the realistic failure, not an attacker.

Q3.6 — "You are sending an artisan's voice to a third-party model. What exactly leaves the country?"

Think. This is the question I most want to be asked, because we have an unusually complete answer and one uncomfortable admission inside it. Lead with the admission.

The admission first. One piece of personal data does go: the artisan's name.

`docs/app/AI-Data-Flow.md` states it in its own words: *"A name is personal data, and for the name question the transcript is the name. There is no way to interpret 'what is your name' without sending it."* It goes **without any identifier attached** — the provider receives a name and nothing to attach it to — **but it goes**. That document exists partly so that nobody discovers this by reading code.

And the consequence we have not closed: the `/consent` notice says we keep the artisan's name. **It does not currently say a third-party processor may see it during onboarding. Under DPDP that is a disclosure that should be made.** It is written down as somebody's job, unassigned.

What is sent — the complete list, four fields, built in the only function that constructs a request body (`build_payload()` in `ai/interpret.py`):

Field	Example	Guard
<code>transcript</code>	<code>"मेरा नाम उत्सव है"</code>	600-char cap, NFKC-normalised, control and invisible characters stripped
<code>question</code>	<code>"onboard.name"</code>	An id , not the text. Must be in <code>KNOWN_QUESTIONS</code> or refused
<code>options</code>	<code>["weaving", "pottery", ...]</code>	Constrained to <code>[a-z0-9_.-]+</code> so this field cannot smuggle prose into the prompt
<code>language</code>	<code>"hi"</code>	One of <code>hi/or/en</code> ; anything else becomes <code>hi</code>

What is never sent — absent from the allowlists, not a policy: artisan id, phone number, auth token, PIN code, the four readiness booleans, consent artifacts, erasure requests, product ids, order data, prices, earnings, and images. **No LLM touches a price.**

Two independent filters, on purpose. `web/api` holds the session — the artisan's id, token and phone are all in scope in that process — and applies `FORWARDED_FIELDS`. `ai/` holds the API key and applies `build_payload`. **Neither trusts the other to have done the filtering, because the cost of one being wrong is personal data leaving the country's jurisdiction with no way to recall it.** There is a self-check in `ai/interpret.py` that passes an artisan id, phone number, token, PIN code and readiness flag into `build_payload` and asserts none of them survive.

The app never holds the API key. A key in a mobile bundle is a published key, and rotating it means an app release rural users never install.

Why the readiness booleans are withheld is worth saying out loud: `has_pan`, `has_bank`, `has_gst`, `has_artisan_card` are **financial-inclusion facts about a named individual**. They are answered by tap, no interpretation is needed, and disclosing them buys nothing. Not sending them is not caution — it is that there was never a reason to.

At scale. The correct end state for a government deployment is **in-country inference**: Bhashini for speech, and an India-hosted open-weights model for interpretation. That removes the name disclosure entirely rather than documenting it. Costing consequences — and the crossover volume where self-hosting a small model becomes cheaper anyway — are in the costing document.

Q3.7 — "Someone talks over the artisan and the microphone hears 'ignore your instructions.' Then what?"

Think. They are testing whether we treat prompt injection as a prompt problem. It is not.

The realistic threat is not a weaver crafting an attack. It is ASR mishearing something that reads as an instruction, a bystander or a television talking over the artisan, or this endpoint later being pointed at a text field somebody can type into.

The system prompt does say to treat the transcript as data. Nothing downstream trusts that it obeyed:

1. The response is parsed as JSON; anything else becomes `null`.
2. For a choice question, **choice must be a member of the caller's options** or it is discarded. A model that returns `"blacksmithing"` when offered eight crafts produces a `null`, however confident it claims to be.
3. Free-text answers are length-capped and control-stripped on the way out.
4. **The model has no tools, no URLs, no ids, no ability to act.** It returns a string.
5. `temperature: 0` — extraction, not writing. The same sentence gives the same answer, or the confirmation step is meaningless.

The property that matters, stated as a property rather than a mitigation:

A model that ignores every instruction it was given can, at worst, return a value the caller already declared acceptable — or nothing.

And after all of it, the artisan sees what we heard and what we made of it, and taps yes or no. That

confirmation is the real backstop, and it is why it must never be optimised away.